Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN :- 22311792

1. Title

Implementation of Tokenization in Natural Language Processing

---

2. Aim

To study and implement **tokenization** as a fundamental text preprocessing step in Natural Language Processing.

---

3. Objectives

- To understand the concept of tokenization in NLP
- To perform **sentence tokenization** and **word tokenization**
- To observe how text is broken into meaningful units
- To prepare textual data for further NLP processing

---

4. Theory

4.1 Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence that focuses on enabling computers to understand, analyze, and process human language.

4.2 Tokenization

Tokenization is the process of **splitting text into smaller units called tokens**. Tokens can be:

- Sentences

- Words
- Subwords or characters

Tokenization is the **first and most important step** in text preprocessing.

4.3 Types of Tokenization (As per the File)

a) Sentence Tokenization

Sentence tokenization divides a paragraph into individual sentences.
It helps in tasks such as sentiment analysis and document summarization.

b) Word Tokenization

Word tokenization splits sentences into individual words or terms.
It is used in feature extraction, text classification, and language modeling.

4.4 Importance of Tokenization

- Converts raw text into processable units
- Improves accuracy of NLP models
- Acts as a base for further steps like stemming, lemmatization, and vectorization

---

5. Program Codes

[1]

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

Word Tokenization using Regular Expression

The code snippet uses the RegexpTokenizer from the Natural Language Toolkit (NLTK) to tokenize a given text based on a regular expression pattern.

[2]

```
from nltk.tokenize import RegexpTokenizer

tokenizer = RegexpTokenizer(r'\w+')
text = "Let’s test it and see how it works."
tokenizer.tokenize(text)
```

```
['Let', 's', 'test', 'it', 'and', 'see', 'how', 'it', 'works']
```

Using split method

[3]

```
sent1 = "NLP is a subfield of computer science. and artificial intelligence."
sent1.split('.')
```

```
['NLP is a subfield of computer science', ' and artificial intelligence', '']
```

[4]

```
import nltk

nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

[6]

```
from nltk.tokenize import word_tokenize

text = "Ready to begin? You’re at GeeksforGeeks!."
word_tokenize(text)
```

```
['Ready',
 'to',
 'begin',
 '?',
 'You',
 '’',
 're',
 'at',
 'GeeksforGeeks',
 '!',
 '.']
```

Whitespace based Tokenization

Syntax : tokenize.WhitespaceTokenizer()

Return : Return the tokens from a string

[9]

```
# import WhitespaceTokenizer() method from nltk
from nltk.tokenize import WhitespaceTokenizer

# Create a reference variable for Class WhitespaceTokenizer
wt = WhitespaceTokenizer()

# Create a string input
text = "Hello there; welcome to GeeksforGeeks. \n Let’s dive in \t"
print("\nOriginal string:")
print(text)
# Use tokenize method
tokenized_text = wt.tokenize(text)
print("\nSplitting using whitespece into separate tokens:")
print(tokenized_text)
```

```
Original string:
Hello there; welcome to GeeksforGeeks.
 Let’s dive in

Splitting using whitespece into separate tokens:
['Hello', 'there;', 'welcome', 'to', 'GeeksforGeeks.', 'Let’s', 'dive', 'in']
```

Punctuation-based tokenizer using WordPunctTokenizer

The WordPunctTokenizer is one of the NLTK tokenizers that splits words based on punctuation boundaries. Each punctuation mark is treated as a separate token.

[13]
✓ 0s

```
from nltk.tokenize import WordPunctTokenizer
text = "Welcome to the VIIT NLP page—every student’s resource.;"
print("\nOriginal string:")
print(text)
result = WordPunctTokenizer().tokenize(text)
print("\nSplit all punctuation into separate tokens:")
print(result)
```

```
Original string:
Welcome to the VIIT NLP page—every student’s resource.;

Split all punctuation into separate tokens:
['Welcome', 'to', 'the', 'VIIT', 'NLP', 'page', '—', 'every', 'student', '’', 's', 'resource', '.;']
```

Treebank Tokenizer

[14]
✓ 0s

```
from nltk.tokenize import TreebankWordTokenizer

#tokenizer = TreebankWordTokenizer()
#tokenizer.tokenize(text)

result = TreebankWordTokenizer().tokenize(text)
print(result)
```

```
['Welcome', 'to', 'the', 'VIIT', 'NLP', 'page—every', 'student’s', 'resource.', ';']
```

Tweet Tokenizer

When we want to apply tokenization in text data like tweets, the tokenizers mentioned above can’t produce practical tokens. Through this issue, NLTK has a rule based tokenizer special for tweets. We can split emojis into different words if we need them for tasks like sentiment analysis.

[15]
✓ 0s

```
from nltk.tokenize import TweetTokenizer

tweet_tokenize = TweetTokenizer()
sample_tweet = "Who is your favourite cryptocurrency influencer? 🗣️🏆 Tag them below! 👇"
print(tweet_tokenize.tokenize(sample_tweet))
```

```
['Who', 'is', 'your', 'favourite', 'cryptocurrency', 'influencer', '?', '🗣', '🏆', 'Tag', 'them', 'below', '!', '👇']
```

Multi-Word Expression Tokenizer

A MWETokenizer takes a string which has already been divided into tokens and retokenizes it, merging multi-word expressions into single tokens, using a lexicon of MWEs.

The multi-word expression tokenizer is a rule-based, “add-on” tokenizer offered by NLTK. Once the text has been tokenized by a tokenizer of choice, some tokens can be re-grouped into multi-word expressions.

For example, the name Steven Spielberg is combined into a single token instead of being broken into two tokens. This tokenizer is very flexible since it is agnostic of the base tokenizer that was used to generate the tokens.

[18]
✓ 0s

```
# import MWETokenizer() method from nltk
from nltk.tokenize import MWETokenizer
#from nltk.tokenize import  word_tokenize

# Create a reference variable for Class MWETokenizer
tokenizer = MWETokenizer([('a', 'little'), ('a', 'little', 'bit'), ('a', 'lot')])

tokenizer.add_mwe(('in', 'spite', 'of'))
print(tokenizer.tokenize('I learned a little and a little bit, and a lot in spite of challenges'.split()))
```

```
['I', 'learned', 'a_little', 'and', 'a_little', 'bit,', 'and', 'a_lot', 'in_spite_of', 'challenges']
```

[19]
✓ 0s

```
from nltk.tokenize import MWETokenizer
tokenizer = MWETokenizer()
tokenizer.add_mwe(('Steven', 'Spielberg'))
tokenizer.tokenize('Steven Spielberg is an American writer producer director '.split())
```

```
['Steven_Spielberg', 'is', 'an', 'American', 'writer', 'producer', 'director']
```

Sentence Tokenization using sent_tokenize

The code snippet uses sent_tokenize function from NLTK library. The sent_tokenize function is used to segment a given text into a list of sentences.

[20]

```
from nltk.tokenize import sent_tokenize

text = "Hi learners. Welcome to our NLP course. Let us begin the lesson."
sent_tokenize(text)
```

```
['Hi learners.', 'Welcome to our NLP course.', 'Let us begin the lesson.']
```

Sentence Tokenization using PunktSentenceTokenizer

When huge amount of data available then it is efficient to use 'PunktSentenceTokenizer' from the NLTK library. The Punkt tokenizer is a data-driven sentence tokenizer included in NLTK. It is trained on large corpus of text to identify sentence boundaries.

[21]

```
import nltk.data

# Loading PunktSentenceTokenizer using English pickle file
tokenizer = nltk.data.load('tokenizers/punkt/PY3/english.pickle')
tokenizer.tokenize(text)
```

```
['Hi learners.', 'Welcome to our NLP course.', 'Let us begin the lesson.']
```

Tokenize sentence of different language

One can also tokenize sentence from different languages using different pickle file other than English. In the following code snippet, we have used NLTK library to tokenize a Spanish text into sentences using pre-trained Punkt tokenizer for Spanish. The Punkt tokenizer is a data-driven tokenizer that uses machine learning techniques to identify sentence boundaries.

[22]

```
import nltk.data

spanish_tokenizer = nltk.data.load('tokenizers/punkt/PY3/spanish.pickle')

text = 'Hola amigo. Estoy bien.'
spanish_tokenizer.tokenize(text)
```

```
['Hola amigo.', 'Estoy bien.']
```

Character tokenization

Character tokenization is a simple tokenization method where a text is split into individual characters rather than words or subwords. Each character becomes a token, including letters, digits, spaces, punctuation, and special characters.

[23]

```
def character_tokenization(text):
    return list(text)

# Example input
text = "Hello, world!"

# Tokenize the text into characters
tokens = character_tokenization(text)

print("Input Text:", text)
print("Character Tokens:", tokens)
```

```
Input Text: Hello, world!
Character Tokens: ['H', 'e', 'l', 'l', 'o', ',', ' ', 'w', 'o', 'r', 'l', 'd', '!']
```

Removal of stop words

The words which are generally filtered out before processing a natural language called stop words.

Examples: "the", "a", "an", "so", "what"

[24]

```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

[25]

```
from nltk.corpus import stopwords
```

[26]

```
stop = stopwords.words('english')
stop
```

```
 'the',
 'their',
 'theirs',
 'them',
 'themselves',
 'then',
 'there',
 'these',
 'they',
 "they'd",
 "they'll",
 "they're",
 "they've",
 'this',
 'those',
```

```
 'their',
 'theirs',
 'them',
 'themselves',
 'then',
 'there',
 'these',
 'they',
 "they'd",
 "they'll",
 "they're",
 "they've",
 'this',
 'those',
 'through',
 'to',
 'too',
 'under',
 'until',
 'up',
 've',
 'very',
 'was',
 'wasn',
 "wasn't",
 'we',
 "we'd",
 "we'll",
 "we're",
 'were',
 'weren',
 "weren't",
 "we've",
 'what',
 'when',
 'where',
 'which',
```

```
 'wasn',
 "wasn't",
 'we',
 "we'd",
 "we'll",
 "we're",
 'were',
 'weren',
 "weren't",
 "we've",
 'what',
 'when',
 'where',
 'which',
 'while',
 'who',
 'whom',
 'why',
 'will',
 'with',
 'won',
 "won't",
 'wouldn',
 "wouldn't",
 'y',
 'you',
 "you'd",
 "you'll",
 'your',
 "you're",
 'yours',
 'yourself',
 'yourselves',
 "you've"]
```

[27]

```
var = 'Natural language processing (NLP) is a subfield of computer science and especially artificial intelligence. It is primarily concerned with providing compu
```

[28]

```
var
```

```
'Natural language processing (NLP) is a subfield of computer science and especially artificial intelligence. It is primarily concerned with providing computers w
ith the ability to process data encoded in natural language and is thus closely related to information retrieval, knowledge representation and computational ling
uistics, a subfield of linguistics. Typically data is collected in text corpora, using either rule-based, statistical or neural-based approaches in machine learn
ing and deep learning.'
```

[29]

```
import nltk

nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

[30]

```
from nltk.tokenize import word_tokenize
var_new = word_tokenize(var)
print(var_new)
```

```
['Natural', 'language', 'processing', '(', 'NLP', ')', 'is', 'a', 'subfield', 'of', 'computer', 'science', 'and', 'especially', 'artificial', 'intelligence', '.',
```

[31]

```
from nltk.corpus import stopwords
from string import punctuation

stop = stopwords.words('english')
print(len(stop))
stop_word_list = list(punctuation)+stop
print(stop_word_list)
```

[31]
0s

```python
from nltk.corpus import stopwords
from string import punctuation

stop = stopwords.words('english')
print(len(stop))
stop_word_list = list(punctuation)+stop
print(stop_word_list)

for i in var_new:
  if i not in stop_word_list:
    print(i)
```

```
198
['!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', '[', '\\', ']', '^', '_', '`', '{', '|', '}', '~',
Natural
language
processing
NLP
subfield
computer
science
especially
artificial
intelligence
It
primarily
concerned
providing
computers
```

```
processing
NLP
subfield
computer
science
especially
artificial
intelligence
It
primarily
concerned
providing
computers
ability
process
data
encoded
natural
language
thus
closely
related
information
retrieval
knowledge
representation
computational
linguistics
subfield
linguistics
Typically
data
collected
text
corpora
using
either
```

```
natural
language
thus
closely
related
information
retrieval
knowledge
representation
computational
linguistics
subfield
linguistics
Typically
data
collected
text
corpora
using
either
rule-based
statistical
neural-based
approaches
machine
learning
deep
learning
```

[32]
0s

```python
punctuation
```

```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

Lemmatization

Lemmatization is the process of reducing words to their base or dictionary form, known as the lemma. For example, a lemmatization algorithm would reduce the word better to its root word, or lemme, good.

[33]

```python
import nltk
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

[34]

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()

sentence = "He was running and eating at the same time. He has bad habit of running, having long hours of work and then eats "
punctuation = "?:!.,;"
sentence_words = nltk.word_tokenize(sentence)
for word in sentence_words:
    if word in punctuation:
        sentence_words.remove(word)

sentence_words
print("{0:20}{1:20}".format("Word","Lemma"))
for word in sentence_words:
  print("{0:20}{1:20}".format(word,wordnet_lemmatizer.lemmatize(word,pos="v")))
```

```
Word                Lemma
He                  He
was                 be
running             run
```

[34]

```python
for word in sentence_words:
    if word in punctuation:
        sentence_words.remove(word)

sentence_words
print("{0:20}{1:20}".format("Word","Lemma"))
for word in sentence_words:
  print("{0:20}{1:20}".format(word,wordnet_lemmatizer.lemmatize(word,pos="v")))
```

```
Word                Lemma
He                  He
was                 be
running             run
and                 and
eating              eat
at                  at
the                 the
same                same
time                time
He                  He
has                 have
bad                 bad
habit               habit
of                  of
running             run
having              have
long                long
hours               hours
of                  of
work                work
and                 and
then                then
eats                eat
```

---

6. Result and Outcomes

Result

- The input text was successfully divided into sentences using sentence tokenization.
- Each sentence was further divided into words using word tokenization.
- The output tokens clearly represent individual linguistic units.

Outcomes

- Understood the working of sentence and word tokenization
- Learned how tokenization prepares text for NLP tasks
- Gained practical experience using NLTK tokenizers
- Built a strong foundation for further text preprocessing techniques

---

7. Conclusion

Tokenization is a crucial preprocessing step in Natural Language Processing. It helps break down raw text into meaningful units, making it suitable for computational analysis. Through this experiment, both sentence and word tokenization were successfully implemented, demonstrating their importance in preparing textual data for advanced NLP applications.